IMT Nord Europe
École Mines-Télécom
IMT-Université de Lille

RÉPUBLIQUE
FRANÇAISE
Liberté
Égalité
Fraternité

Quantum Generative Adversarial Networks Hybrid Architectures and Convergence Analysis

Leveraging Variational Quantum Circuits for High-Dimensional Data Synthesis

Antony Manuel

IMT Nord Europe
Department of Computer Science
Master of Data Science

Lille, April 2026

Abstract

This report presents the design, implementation, and iterative refinement of a hybrid quantum-classical generative adversarial network for image synthesis, following the Patch-Quantum Wasserstein GAN (PQWGAN) architecture trained with a gradient-penalty objective (WGAN-GP) from Tsang et al.'s work [1]. The generator is composed of 16 independent parameterized quantum circuit (PQC) subgenerators, each implemented in PennyLane with a 5-qubit ansatz (4 data qubits plus 1 ancilla) and a layered structure of trainable single-qubit rotations interleaved with nearest-neighbor entangling gates. Each subgenerator produces a 4x4 pixel patch via ancilla-conditioned measurement probabilities, which are assembled into full 16x16 grayscale images. The discriminator is a classical convolutional (or fully-connected) critic network trained under the WGAN-GP objective, augmented with a total variation regularization term to encourage spatial coherence across independently generated patches. The system is evaluated on a binarized two-class subset of MNIST downsampled to 16x16 resolution.

The report documents these findings, the corrective patches applied, and concrete recommendations for further improving training stability, reproducibility, and quantitative evaluation of the resulting quantum GAN.

Keywords: Quantum computing, Parameterized Quantum Circuits, Generative Adversarial Networks, Quantum Machine Learning, Quantum Generative Adversarial Networks, Training Stability

Notation

$\mathcal{X}$	Set.
$\mathbb{N}$	Set of integers.
$\mathbb{R}$	Set of real numbers.
$\subset$	Strict inclusion.
$\subseteq$	Inclusion with possible equality.
$\lvert \cdot \rvert$	Absolute value of a real number.
$\lVert \cdot \rVert$	Euclidean norm of a vector.
x	A real scalar belonging to $\mathbb{R}$.
$x \in \mathbb{R}^n$	A vector in $\mathbb{R}^n$.
$X \in \mathbb{R}^{n \times d}$	A matrix in $\mathbb{R}^{n \times d}$.
I_n	Identity matrix of size n.
$X^\top$	Transpose of X.
X^{-1}	Inverse of X.
x	A random scalar variable.
$x \in \mathbb{R}^n$	A random vector in $\mathbb{R}^n$.
$X \in \mathbb{R}^{n \times d}$	A random matrix in $\mathbb{R}^{n \times d}$.
$P(x)$, $P(\mathbf{x})$	Probability distribution of x (resp. joint probability distribution of $\mathbf{x}$).
$x \sim P_x$	Random variable x follows a probability distribution P_x.
$\mathbb{E}(x)$	Expectation of x with respect to $P(x)$.
$\mathrm{Var}(x)$	Variance of x under $P(x)$.
x_i	i-th element of the vector x.
$X_{i,j}$	Element at row i, column j of the matrix X.
$X_{i,:}$	i-th row of the matrix X.
$X_{:,i}$	i-th column of the matrix X.
$f \circ g$	Composition of functions f and g.
dy/dx	Derivative of y with respect to x.
$\partial y/\partial x$	Partial derivative of y with respect to x.
$\partial f/\partial x$	Jacobian matrix $J \in \mathbb{R}^{n \times p}$ of $f : \mathbb{R}^n \to \mathbb{R}^p$.

Sometimes we use a function f whose argument is a scalar but apply it to a vector $f(x)$ or a matrix $f(X)$. This denotes the element-wise application of f to the array.

$$* \quad * \quad *$$

Contents

Contents **3**

1 Introduction **5**

2 Background **7**
1 Generative Modeling 7
1.1 Generative Adversarial Networks 7
1.2 Wasserstein Generative Adversarial Networks 8
2 Quantum computing 8
2.1 Quantum Bits 9
2.2 Quantum gates and circuits 10
2.2.1 Single Qubits Gates 10
2.2.2 Multi Qubit Gates 10
2.3 Variational Quantum Algorithms 11

3 QGAN Framework **13**
1 Architecture Overview 13
1.1 Generator Logic 13
1.2 Critic structure 14
2 Training Strategy 15
2.1 Training objective 15
2.2 Training algorithm 15
2.3 Dataset and Libraries 16
2.4 Hyperparameters 16

4 Results **18**
1 Results Analysis 18
1.1 Effect of Entangling Gate choice (CNOT vs. CZ) 18
1.2 Effect of the Gradient-Penalty Coefficient 19
1.3 Effect of the Generator Learning Rate 19

5 Conclusion **21**

Bibliography **23**

Annexes **24**

A Appendix **25**

1 Details on generator hyperparameters effects. 25
2 Differences from the different ansatz 25

B Annexe **27**
1 Cost Functions 27
1.1 Critic Loss with Gradient Penalty 27
1.2 Generator Loss with Total Variation Regularization 27
2 Measurement and Quantum Expectation 27
3 Gradient Descent 28
3.1 Parameter-Shift Rule 28
4 Derivation of the Post-Measurement Density Matrix 28
4.1 Projective Measurements on Composite Systems 28
4.2 Post-Measurement State 29
4.3 Reduced Density Matrix of the Data Register 29

Introduction

Over the past decade, generative models have emerged as one of the most dynamic areas of development within deep learning, continually finding novel applications across diverse domains, including image synthesis, musical composition, and drug discovery. Spanning from Variational Autoencoders (VAEs) to autoregressive models, the field of generative modeling has yielded a plethora of architectures and techniques. Each paradigm possesses distinct strengths and limitations, collectively pushing the boundaries of machine-driven creation. Among these varied approaches, Generative Adversarial Networks (GANs), introduced in 2014 [2], have stood out for their remarkable capacity to generate highly realistic data. GANs have been successfully deployed across a wide spectrum of applications, such as image synthesis [2], text generation, molecular and physical data simulation, and 3D object generation. However, the deployment of classical GANs is constrained by several significant challenges, notably mode collapse, vanishing gradients, extreme sensitivity to hyperparameter selection, and an immense demand for both data and computational resources. Numerous architectural and algorithmic enhancements have been proposed to mitigate these issues [3], for instance Wasserstein GANs [4] employ a distinct distance metric to formulate a loss function that improves training stability, while Karras et al. [5] proposed to incrementally scales the resolution of generated images during the training phase. Despite these advancements, training classical GANs remains inherently complex and computationally expensive, frequently necessitating exhaustive empirical tuning to achieve optimal convergence.

Concurrently, quantum computing has emerged as a transformative computational paradigm, holding the promise of solving certain classes of problems that are intractable for classical computers, such as Shor's algorithm for prime factorization and Grover's algorithm for unstructured database search. Driven by these theoretical advantages, there has been a burgeoning interest in exploring the intersection of quantum computing and machine learning. This synthesis has catalyzed the development of Quantum Machine Learning (QML) algorithms, which aim to harness quantum speedups for complex pattern recognition and optimization tasks. Within the QML landscape, extending quantum methodologies to generative modeling presents a particularly compelling frontier, culminating in the conceptualization of Quantum Generative Adversarial Networks (QGANs) [6, 7]. By replacing traditional neural networks with parameterized quantum circuits (PQCs) [8] within the generator, the discriminator, or both, QGANs theoretically possess greater representational capacity. This quantum advantage could enable the learning of highly complex, high-dimensional probability distributions more efficiently than classical frameworks, potentially alleviating the severe computational bottlenecks inherent to traditional GAN training.

In this work, we consider the following research papers [1, 9, 10] which propose novel

QGAN architectures and training methodologies that aim to enhance the stability and performance of quantum generative models.

Background

1 Generative Modeling

In machine learning, generative modeling is a type of unsupervised learning focused on learning the underlying distribution of a dataset to generate new samples that resemble the original data. Generative models can be broadly categorized into explicit and implicit frameworks. Explicit generative models aim to define or approximate the explicit probability distribution of the data, allowing for likelihood evaluation and direct sampling; examples include Variational Autoencoders (VAEs) [11] and diffusion models [12]. Conversely, implicit generative models, such as Generative Adversarial Networks (GANs) [2], learn to generate realistic samples through adversarial training without explicitly modeling the underlying probability density function. These models span a wide range of domains, including image synthesis, text generation, and data augmentation, and are frequently utilized in financial environments, healthcare, and molecular biology. Because GANs are the primary focus of this research, the following sections provide a detailed overview of their architecture, training process, and subsequent architectural improvements.

1.1 Generative Adversarial Networks

Introduced by Goodfellows et al.[2], Generative Adversarial Networks are models able to learn the distributed probabilities underlying training data and generate new data based on that probability. Moreover GANs are really usefull for understanding distribution probability in unlabeled data, and generate data that are not present in the initial training set.

Basically, a GAN consists of two models, a generator G that outputs synthetic samples $\hat{x}$ given a noise variable input z, bringing potential diversity in the generating process, and is trained to capture the real data distribution in order to generate samples as close as possible to train data. The second model is a discriminator D that estimates the probability of a given sample x being synthetic or coming from the real dataset.

This system inspired from the game theory implies two scenario. The first one emphasizes the goal of the discriminator to classify the input x as real or fake by minimizing the difference between its output probability $D(x) = P(x|y)$ and the true label y. The second scenario appears when noise inputs z to the generator are randomly sampled from the model's prior over the latent variables, then the discriminator receives input $G(z) = \hat{x}$, a fake sample created by the generator, then the discriminator strives to make $D(\hat{x})$ approach 0, while the generator strives to make the same quantity approach 1.

At the end, if both models have sufficient abilities, then the game reached the Nash equilibrium, in other words, it's corresponding to P_G, the learned distribution matching P_{data} the real then D having a probability $1/2$ of correctly classifying real and fake samples, which means G is able to produce samples indistinguishable from the real data, and D is unable to differentiate between them.

The loss of the model is based on two components, the generator loss and the discriminator loss

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{data}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(\hat{x}))] \tag{2.1}$$

1.2 Wasserstein Generative Adversarial Networks

As previously mentioned, training classical GANs is notoriously difficult, frequently suffering from mode collapse, vanishing gradients, and extreme sensitivity to hyperparameter configurations. To mitigate these pathologies, Arjovsky et al. [4] proposed the Wasserstein GAN (WGAN), which substitutes the traditional Jensen-Shannon divergence with the Wasserstein distance. To formalize this approach while ensuring training stability, Gulrajani et al. [13] introduced the WGAN with Gradient Penalty (WGAN-GP). Under this framework, the Discriminator no longer classifies inputs with a probability $p \in [0,1]$; instead, it acts as a *critic* D that outputs a scalar score representing the *realness* of the data. Therefore, the objective function of WGAN-GP is formulated as:

$$\min_G \max_{D\in\mathcal{D}} V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}}[D(x)] - \mathbb{E}_{z\sim p_z}[D(G(z))] - \lambda \mathbb{E}_{\hat{x}\sim p_{\hat{x}}}\left[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2\right] \tag{2.2}$$

where the first two terms compute the original Wasserstein distance, and the third term represents the gradient penalty weighted by the hyperparameter λ. The penalty is evaluated on interpolated samples $\hat{x} = \epsilon x + (1-\epsilon)G(z)$, where $\epsilon \sim \mathcal{U}[0,1]$. This penalty enforces a 1-Lipschitz constraint on the Critic by penalizing deviations of the gradient norm from 1, effectively preventing the gradients from vanishing or exploding.

Incorporating the WGAN-GP framework is particularly critical when designing a patch-based Quantum GAN (QGAN) for generating structural data like 28×28 MNIST digits. First, the Lipschitz constraint regularizes the critic's landscape, preventing its gradients from becoming too steep, which directly mitigates the severe issue of barren plateaus (narrow gradients) inherent to Parameterized Quantum Circuits (PQCs). Second, since a patch-based architecture generates independent sub-regions of the image, a standard GAN loss often struggles with spatial coherence, resulting in a disjointed, mosaic-like final output. By providing a continuous real-valued score instead of a binary classification, the WGAN-GP critic offers smoother and more informative gradient feedback. This guides the quantum generator to effectively learn global spatial correlations across boundaries, ensuring seamless integration of the patches.

2 Quantum computing

This section provides a brief overview of the fundamental concepts of quantum computing. It begins with an introduction to quantum information and the representation of quantum states, followed by the description of quantum gates and circuits. Finally, it introduces the concept of Variational Quantum Algorithms (VQAs).

2.1 Quantum Bits

In quantum physics, each quantum state ψ of each quantum system can be represented by a ket $|\psi\rangle$, up to a global phase $e^{i\varphi}$, which is a vector of an Hilbert space $\mathcal{H}$ with its norm equal to 1.

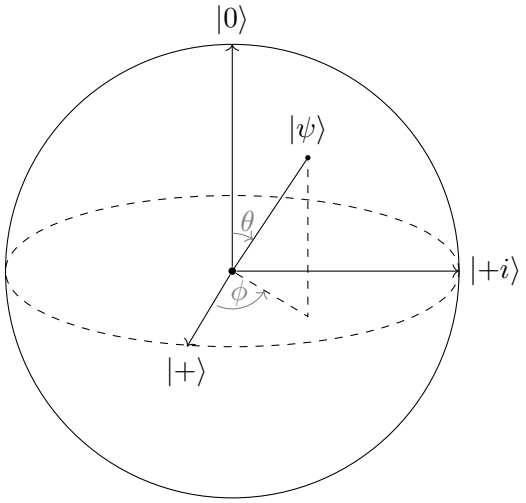


Fig. 2.1: Bloch Sphere

Unlike a classical bit, which is restricted to a binary state $b \in \{0, 1\}$, a qubit exists as a unit vector in a two-dimensional complex Hilbert space $\mathcal{H} \cong \mathbb{C}^2$. Using the computational basis $\mathcal{B} = \{|0\rangle, |1\rangle\}$, a general state $|\psi\rangle$ is expressed as a linear combination:

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle \tag{2.3}$$

where $\alpha, \beta \in \mathbb{C}$ are probability amplitudes. According to the Born rule, the probability of measuring the state in $|0\rangle$ is $|\alpha|^2$ and in $|1\rangle$ is $|\beta|^2$, requiring the normalization condition $|\alpha|^2 + |\beta|^2 = 1$. Let's write the amplitudes in polar form, such as $\alpha = ae^{i\varphi}$ and $\beta = be^{i\gamma}$, where a, $b \in \mathbb{R}$. Hence, we can rewrite $|\psi\rangle$ as

$$|\psi\rangle = e^{i\varphi}(a\,|0\rangle + be^{(\gamma-\varphi)}\,|1\rangle) \tag{2.4}$$

since the global phase, $e^{i\varphi}$, does not affect the outcome measurement probabilities, and that 2.4 describe the exact same state :

$$|\psi\rangle = a\,|0\rangle + be^{\phi}\,|1\rangle \tag{2.5}$$

Additionally, the normalization of $|\psi\rangle$ requires that $a^2 + b^2 = 1$ in the way, we can parameterize the state using two real angles θ and ϕ:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle \tag{2.6}$$

where $\theta \in [0, \pi]$ represents the latitude and $\phi \in [0, 2\pi)$ the longitude. As a result, the state of a single qubit can be visualized as a vector in the Bloch sphere 2.1.

2.2 Quantum gates and circuits

In quantum computing, the manipulation of qubit states is executed through quantum circuits, i.e., computational models consisting of a sequence of quantum gates applied to a register of qubits. Mathematically, these gates are represented by unitary operators U (where $U^\dagger U = I$), a property that preserves the norm of the quantum state, thereby ensuring that the transformation is completely reversible and the total probability strictly remains equal to 1.

2.2.1 Single Qubits Gates

As the qubits evolve within the circuit, their state transformation mathematically corresponds to multiplying the state vector by the unitary matrix. Several gates are commonly used, such as the Pauli gates (X, Y, Z) and the Hadamard gate (H):

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Consequently, the transition from an initial state $|\psi_0\rangle$ to a final state $|\psi_f\rangle$ is governed by the equation $|\psi_f\rangle = U\,|\psi_0\rangle$, where U denotes the unitary operator corresponding to the sequence of gates applied within the circuit.

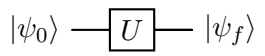


Fig. 2.2: Representation of a gate U applied to a state $|\psi_0\rangle$ resulting in a state $|\psi_f\rangle$.

2.2.2 Multi Qubit Gates

While single-qubit gates enable transformations on individual qubits, multi-qubit gates are fundamental for creating entanglement and establishing quantum correlations across composite quantum systems. The state space of an n-qubit system is described by the tensor product of the individual Hilbert spaces, $\mathcal{H} = \mathcal{H}_1 \otimes \mathcal{H}_2 \otimes \cdots \otimes \mathcal{H}_n \cong (\mathbb{C}^2)^{\otimes n}$, of dimension 2^n.

Multi-qubit operations are represented by $2^n \times 2^n$ unitary matrices acting on the composite state. The most prominent two-qubit gate is the Controlled-NOT (CNOT) gate (or CX gate). It acts on a control qubit and a target qubit, flipping the state of the target qubit (applying a Pauli-X operation) if and only if the control qubit is in the state $|1\rangle$:

$$\text{CNOT} = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes X = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \tag{2.7}$$

When applied to a superposition state, such as $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |0\rangle$, the CNOT gate generates the maximally entangled Bell state:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \tag{2.8}$$

Other commonly utilized two-qubit operations include the Controlled-Z (CZ) gate, the SWAP gate, and various parameterized controlled rotations.
The Controlled-Z (CZ) gate applies a phase flip to the target qubit if the control qubit is in the state $|1\rangle$, represented by the matrix:

$$\mathrm{CZ} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \tag{2.9}$$

The Swap gate exchanges the quantum states of two qubits:

$$\mathrm{SWAP} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{2.10}$$

Additionally, Controlled Phase (CR_ϕ) and Controlled Rotation gates implement continuous, parameterized multi-qubit interactions conditioned on the state of the control register.

In the context of Parameterized Quantum Circuits (PQCs) and Quantum Generative Adversarial Networks (QGANs), multi-qubit entangling gates are arranged in structured patterns (e.g., circular, nearest-neighbor, or all-to-all topologies) between parameterized rotation layers. These entangling layers enable the circuit to learn and express complex, high-dimensional non-local probability distributions that cannot be factored into independent marginal distributions.

2.3 Variational Quantum Algorithms

In addition to these basic gates, parameterized rotation gates $R_x(\theta)$, $R_y(\theta)$, and $R_z(\theta)$ enable arbitrary rotations on the Bloch sphere, as depicted in Fig. 2.3.

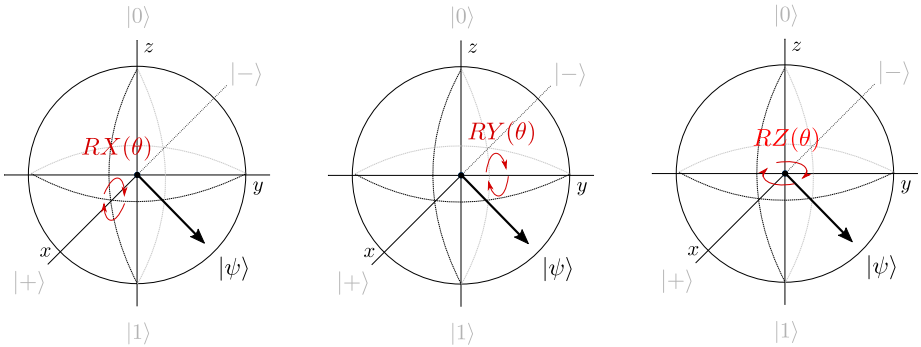


Fig. 2.3: Rotation gates on the Bloch sphere.

Note. Illustration from Pennylane codebook more reference here.

Therefore, a Variational Quantum Circuit (VQC), also known as an ansatz, incorporates these parameterized gates. The underlying parameters are optimized using classical optimization algorithms to minimize a cost function. This hybrid quantum-classical approach, through Variational Quantum Algorithms (VQAs)[14], allows us to leverage the computational advantages of quantum circuits while utilizing classical resources for optimization,

which represents a cornerstone approach in quantum machine learning and quantum optimization.

In the context of a QGAN, the generator typically relies on such parameterized quantum circuits. It applies rotation gates to steer the state vector across the Bloch sphere, aiming to align the output with the target data distribution. The ultimate objective is to optimize these parameters so that the resulting quantum measurement statistics become indistinguishable from the real data distribution provided to the discriminator.

QGAN Framework

1 Architecture Overview

This work builds on the WGAN-GP framework [4, 13], which demonstrated significant improvements in GAN training stability. Recent work on quantum generative adversarial networks [1, 9] has adopted a similar setup, replacing the classical generator with a quantum one while retaining a classical critic [10]. In the following sections, we provide an overview of the architecture of the generator and the critic.

1.1 Generator Logic

The generator architecture follows the same structure as in [10] and [1]. It is composed of N_S so-called sub-generators, each implemented as a variational quantum circuit (VQC). Each sub-generator takes a latent vector $z \in \mathbb{R}^N$ as input, where N denotes the number of qubits, and encodes it into a quantum state via R_Y gates. Starting from the state $|0\rangle^{\otimes N}$, this encoding yields the latent state

$$|z\rangle = \bigotimes_{i=1}^{N} R_Y(z_i) \, |0\rangle^{\otimes N} . \tag{3.1}$$

The latent state is then processed by a variational quantum circuit consisting of L layers of parameterized gates. This circuit implements a unitary operator $U(\theta, \phi, \omega)$ acting on $|z\rangle$ to produce the output state

$$|\psi_k\rangle = U(\theta, \phi, \omega) \, |z\rangle . \tag{3.2}$$

Each of the L layers consists of the universal single-qubit gate $R(\theta, \phi, \omega) = R_Z(\theta) R_Y(\phi) R_Z(\omega)$, followed by CNOT or CZ gates to generate entanglement between qubits. The $R(\theta, \phi, \omega)$ gate is chosen for its ability to generate an arbitrary single-qubit rotation and, owing to its decomposition into a sequence of Z-Y-Z rotations, for its ease of implementation on real quantum devices. Each sub-generator can therefore be defined as a unitary operator $U_S(\theta, \phi, \omega, z)$ parameterized by the latent vector z and the gate parameters $\theta, \phi, \omega \in \mathbb{R}^{N \times L}$.

$$U_S(\theta, \phi, \omega, z) = U(\theta, \phi, \omega) \bigotimes_{i=1}^{N} R_Z(z_i) \tag{3.3}$$

with U defined as follows:

$$U(\theta, \phi, \omega) = \prod_{l=1}^{L} E_l \left(\bigotimes_{i=1}^{N} R_Z(\theta_{l,i}) R_Y(\phi_{l,i}) R_Z(\omega_{l,i}) \right) , \tag{3.4}$$

Moreover, replicating non-linear behavior in the generator is essential, since a standard quantum circuit, including the circuits defined above, implements only linear transformations. Introducing non-linearity into the generator is therefore necessary. Several ansätze have been proposed in the literature; we focus on those introduced by [10] and [1], which share the same underlying structure and differ only in their controlled gate. Both approaches achieve non-linearity through the same mechanism: the N qubits are split into A ancilla qubits and D data qubits, and a partial measurement M is performed on the ancilla register. Following [1], this measurement is defined as a projection of the ancilla qubits onto the all-zero state,

$$M = (|0\rangle \langle 0|)^{\otimes A}. \tag{3.5}$$

The state of the remaining D qubits is then isolated by tracing out the ancilla register with the partial trace operator Tr_A, yielding the density matrix

$$\rho_D = \mathrm{Tr}_A \left(\frac{(|0\rangle \langle 0|)^{\otimes A} \otimes \mathbb{I} \ |\psi_i\rangle \langle \psi_i|}{\langle \psi_i| (|0\rangle \langle 0|)^{\otimes A} \otimes \mathbb{I} \, |\psi_i\rangle} \right). \tag{3.6}$$

The measurement outcomes are then post-processed as $G_k(z) = \left[p_0^{(0)}, p_1^{(0)}, \ldots, p_{2^D-1}^{(0)}\right]$, where $p_k^{(0)}$ denotes the probability of measuring the ancilla qubits in state $|0\rangle$ conditioned on the k-th data qubit configuration. This distribution is subsequently normalized as

$$G'_k(z) = \frac{G_k(z)}{\max\left(G_k(z)\right)}. \tag{3.7}$$

in order to have valid pixel value.

Algorithm 1: Algorithm to generate an image from the quantum generator.

Input: Number of ancilla qubits A, number of data qubits D, number of sub-generator N_S, number of layers L, generator parameters $\theta = [\theta_1, \ldots, \theta_P]$, latent variable z.

for $i = 1, \ldots, N_S$ **do**
- $|\psi_i\rangle \leftarrow U_{i,L} \, |z\rangle$;
- $\rho_D \leftarrow \mathrm{Tr}_A \left(\frac{(|0\rangle\langle 0|)^{\otimes A} \otimes \mathbb{I} |\psi_i\rangle\langle\psi_i|}{\langle\psi_i|(|0\rangle\langle 0|)^{\otimes A} \otimes \mathbb{I}|\psi_i\rangle} \right)$;
- measure ρ_D in computational basis to obtain $G_i(z) \leftarrow [p(0), \ldots, p(2^D - 1)]$;
- $G'_i(z) \leftarrow \frac{G_i(z)}{\max(G_i(z))}$;

$G(z) \leftarrow [G'_1(z), \ldots, G'_P(z)]^T$;

return $G(z)$

1.2 Critic structure

Three architectures have been proposed in the literature for the critic: a classical neural network, as used in WGAN-GP [13], a classical convolutional neural network (CNN), and a quantum circuit. However, training the quantum critic proves more challenging due to the barren plateau problem [15], which significantly hinders convergence. Moreover, a CNN architecture does not appear necessary for a dataset as simple as MNIST. A classical neural network is therefore chosen for the critic's architecture, composed of three layers followed by a Leaky ReLU activation with a slope of 0.2.

2 Training Strategy

After defining the architecure of the generator and the critic, we will now discuss the training strategy, including the objective function, the training algorithm, and the experimental setup.

2.1 Training objective

The training objective of the Patch-Quantum Wasserstein GAN (PQWGAN) is formulated under the Wasserstein GAN with Gradient Penalty (WGAN-GP) framework, augmented with a Total Variation (TV) regularization term to promote spatial coherence across independently generated sub-patches.

Critic Objective The classical critic D is trained to maximize the Wasserstein distance while enforcing a 1-Lipschitz continuity constraint via a gradient penalty. The loss function minimized by the critic parameters is defined as:

$$\mathcal{L}_D = \mathbb{E}_{z\sim p_z}\left[D(G(z))\right] - \mathbb{E}_{x\sim p_{\text{data}}}\left[D(x)\right] + \lambda_{\text{gp}}\,\mathbb{E}_{\hat{x}\sim p_{\hat{x}}}\left[\left(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1\right)^2\right], \tag{3.8}$$

where $\hat{x} = \epsilon x + (1-\epsilon)G(z)$ with $\epsilon \sim \mathcal{U}[0,1]$ represents points sampled along straight lines between real data samples x and generated samples $G(z)$, and $\lambda_{\text{gp}} = 10.0$ is the gradient penalty coefficient.

Generator Objective The quantum generator G aims to maximize the critic's score for generated samples (or equivalently, minimize its negative expectation) while minimizing boundary discontinuities across adjacent patches via total variation regularization:

$$\mathcal{L}_G = -\mathbb{E}_{z\sim p_z}\left[D(G(z))\right] + \lambda_{\text{tv}}\,\mathcal{L}_{\text{tv}}(G(z)), \tag{3.9}$$

where $\lambda_{\text{tv}} = 0.1$ denotes the total variation regularization coefficient, and $\mathcal{L}_{\text{tv}}$ penalizes spatial variance across adjacent pixels in the assembled image:

$$\mathcal{L}_{\text{tv}}(X) = \sum_{i,j}\left(|X_{i+1,j} - X_{i,j}| + |X_{i,j+1} - X_{i,j}|\right). \tag{3.10}$$

2.2 Training algorithm

The training procedure adopted in this study aligns with the methodology proposed by Tsang et al. [1], which is predicated on the WGAN-GP framework. Moreover, to evaluate the exact gradients required for loss optimization, we employ the parameter-shift rule for the quantum generator. Subsequent parameter updates for both the generator and the critic are executed using the Adam optimization algorithm. Assuming a generator comprising N_S sub-generators characterized by p parameters, the partial derivative of the generator loss, $\mathcal{L}(\omega,\theta)$, with respect to an individual parameter $\theta_{i,j}$ is analytically evaluated as follows:

$$\frac{\partial\mathcal{L}(\omega,\theta)}{\partial\theta_{i,j}} = \frac{1}{2}\left(\mathcal{L}\left(\omega, \left[\theta_{1,1},\ldots,\theta_{i,j}+\frac{\pi}{2},\ldots,\theta_{N_S,p}\right]\right) - \mathcal{L}\left(\omega, \left[\theta_{1,1},\ldots,\theta_{i,j}-\frac{\pi}{2},\ldots,\theta_{N_S,p}\right]\right)\right). \tag{3.11}$$

Algorithm 2: QGAN training algorithm.

Input: Gradient penalty coefficient λ, critic iterations n_C, number of epochs n_{epochs}, batch size m, Adam hyperparameters $\eta_1, \eta_2, \beta_1, \beta_2$, critic parameters w, geenrator parameters θ.

initialise critic parameters ω, sub-generator parameters θ;
for $epoch = 1, \ldots, n_{epochs}$ **do**
  for $i = 1, \ldots, m$ **do**
    for $t = 1, \ldots, n_C$ **do**
      $x \sim \mathbb{P}_{\text{data}}$, $z \sim p_z$, $\epsilon \sim U[0,1]$;
      $x' \leftarrow \text{quantum_generator}(\theta, z)$;
      $\hat{x} \leftarrow \epsilon x + (1-\epsilon)x'$;
      $L_D^{(i)} \leftarrow D(x') - D(x) + \lambda_{\text{gp}}(||\nabla_{\hat{x}} D(\hat{x})||_2 - 1)^2$;
      $w \leftarrow \text{Adam}\left(\nabla_w \frac{1}{m} \sum_{i=1}^{m} L_D^{(i)}, w, \eta_1, \beta_1, \beta_2\right)$;
    $z \sim p_z$;
    $x' \leftarrow \text{quantum_generator}(\theta, z)$;
    $L_G^{(i)} \leftarrow -D(x') + \lambda_{\text{tv}} \mathcal{L}_{\text{tv}}(x')$;
  $\theta \leftarrow \text{Adam}\left(\nabla_\theta \frac{1}{m} \sum_{i=1}^{m} L_G^{(i)}, \theta, \eta_2, \beta_1, \beta_2\right)$;

2.3 Dataset and Libraries

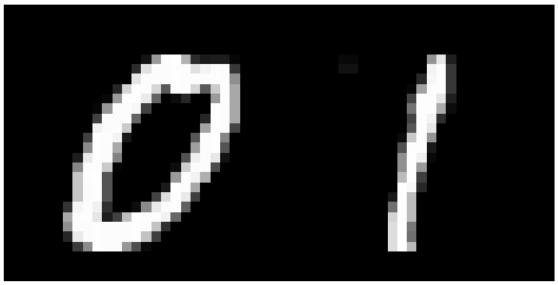


Fig. 3.1: Sample images from the MNIST dataset.

Experiments are conducted on the MNIST dataset, which consists of 28×28 grayscale images of handwritten digits. For simplicity, the images are resized to 16×16 pixels. The quantum circuits and training algorithms are implemented in Python 3, using the Penny-Lane library [16], which provides a high-level interface for quantum machine learning and supports a variety of quantum hardware backends, together with the PyTorch library [17] for classical optimization and data handling.

2.4 Hyperparameters

Table 3.1 summarizes the hyperparameters used for training. Both the generator and the critic are optimized using the Adam optimizer, with distinct learning rates and shared momentum coefficients β_1 and β_2.

Hyperparameter	**Value**
Generator learning rate (η_r)	0.005
Critic learning rate (η_r)	0.0002
Adam β_1	0.0
Adam β_2	0.9
Number of sub-generators (N_S)	16
Qubits per sub-generator (N)	5
Number of layers (L)	2
Gradient penalty coefficient (λ_{gp})	10.0
Total variation loss coefficient (λ_{tv})	0.1

Tab. 3.1: Summary of training hyperparameters.

Results

1 Results Analysis

Although the current implementation did not ultimately achieve the accurate generation of legible digits, it nonetheless yielded valuable insights into minimizing the statistical distance between the generated and target data distributions. Consequently, we conducted a series of empirical experiments to evaluate the impact of various hyperparameters on the training dynamics. Specifically, we investigated the influence of the generator's learning rate, the gradient penalty coefficient.

1.1 Effect of Entangling Gate choice (CNOT vs. CZ)

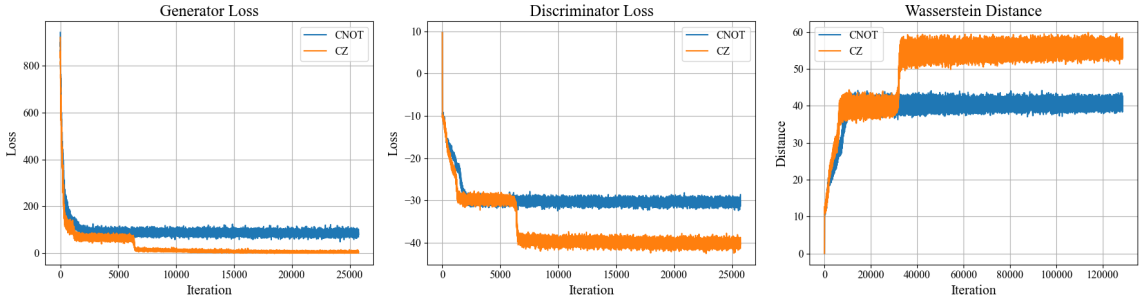


Fig. 4.1: Evolution of the generator loss under various learning rates during training.

The three training-curve panels reveal a consistent divergence between the two entangling-gate choices that begins around iteration 6,000-7,000. Up to that point, CNOT and CZ track each other closely: both generator losses collapse from their initial 900-950 down to roughly 50-100 within the first 1,500 iterations, and both discriminator (critic) losses fall from 10 to a stable band around -28 to -30.

Past this inflection point, the two configurations separate sharply. With CZ, the generator loss continues to collapse toward near-zero, while the critic loss drops further to roughly -38 to -42. With CNOT, both losses instead plateau, generator loss around 80-100, critic loss around -29 to -31, and remain essentially flat for the rest of training. Taken at face value, CZ's lower loss magnitudes might look like better convergence, but the Wasserstein-distance panel indicates the opposite: after a corresponding jump near iteration 30,000, CZ settles at a distinctly *higher* plateau (55-58) than CNOT (40-43), meaning the CZ-based generator's output distribution is actually farther from the real data distribution.

This is consistent with a generator starvation dynamic rather than genuine convergence: a near-zero generator loss alongside an increasingly negative critic loss is the classic WGAN

signature of the critic overpowering the generator, past the point where the generator receives a useful gradient signal. The CNOT ansatz appears to sustain a more balanced (if less dramatic-looking) adversarial equilibrium, and this translates into a measurably better, which means a lower, Wasserstein distance.

1.2 Effect of the Gradient-Penalty Coefficient

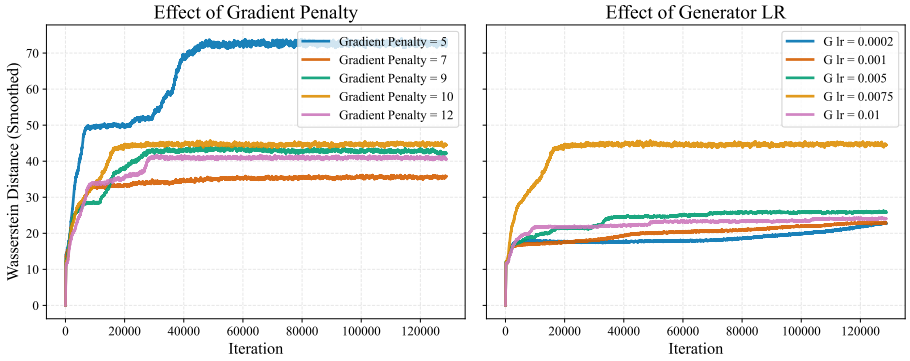


Fig. 4.2: Evolution of the generator loss under various learning rates during training.

Across the gradient penalty coefficients $GP \in \{5, 7, 9, 10, 12\}$, the smoothed Wasserstein distance exhibits three distinct regimes of behavior. The setting $GP = 5$ increases most rapidly, exhibiting a two-stage ascent: an initial rise to approximately 50 by iteration 10,000, followed by a second increase to approximately 72–73 by iteration 50,000, after which it plateaus at this elevated level for the remainder of training. This behavior is characteristic of an under-regularized critic: a weak Lipschitz constraint permits arbitrarily large score gaps, such that the resulting distance estimate likely overstates the true discrepancy between distributions rather than reflecting a genuinely inferior generator.

By contrast, $GP = 7$ yields the most stable trajectory and the lowest terminal value among all five settings, converging to approximately 35–36 within 15,000 iterations and remaining stable thereafter. The settings $GP = 9$, 10, and 12 converge to a common band of approximately 40–45 by iterations 20,000–30,000, with $GP = 10$ positioned marginally above the other two; these three settings remain nearly indistinguishable from one another over the subsequent 100,000 iterations.

Taken together, these results suggest that training is fairly robust to the precise choice of GP once it exceeds approximately 9. However, $GP = 5$ is clearly insufficient and should be excluded from the viable operating range, while $GP = 7$ produces the most favorable trajectory observed and merits consideration as the preferred setting. This latter conclusion should nonetheless be qualified: because under-regularized critics (e.g., $GP = 5$) inflate the estimated distance independently of generator quality, absolute comparisons of this metric across settings are not fully reliable and should be interpreted with caution.

1.3 Effect of the Generator Learning Rate

The learning-rate sweep exhibits a similar pattern, comprising an aggressive outlier and a broadly converging cluster of the remaining settings. The setting $lr = 0.0075$ rises fastest and plateaus earliest and highest, reaching approximately 44–45 by iteration 20,000,

mirroring the behavior observed for $GP = 5$ above. This is likely attributable to an overly aggressive generator update step relative to the critic, and should similarly be interpreted as a sign of training instability rather than as evidence of a superior generator.

The settings $lr = 0.005$ and $lr = 0.01$ both rise to approximately 18–22 early in training. Thereafter, $lr = 0.005$ drifts gradually upward to approximately 25–26 by the end of training, while $lr = 0.01$ plateaus near 22–23. The settings $lr = 0.0002$ and $lr = 0.001$ are nearly indistinguishable from one another for the majority of training, rising slowly and remaining the lowest of all settings (approximately 18–20) until roughly iteration 80,000–100,000, after which both drift upward and converge toward the same 22–23 band occupied by the mid-range rates by the end of the run.

The lower learning rates thus yield the most stable and lowest early-training distance. However, the late-training convergence of nearly all rates toward a similar band, with the exception of the $lr = 0.0075$ outlier, suggests that, given sufficient training iterations, the choice of generator learning rate matters less than early-training behavior alone would indicate. The principal exception is that excessively high learning rates (e.g., $lr = 0.0075$) should be avoided, consistent with the finding for $GP = 5$ discussed above.

Both sweeps exhibit the same qualitative pattern: in each case, one extreme setting ($GP = 5$ and $lr = 0.0075$, respectively) produces a rapidly rising, elevated Wasserstein-distance trace that is more plausibly interpreted as a symptom of training instability than as a faithfully measured distributional gap, since the WGAN distance estimate is meaningful only insofar as the critic satisfies its Lipschitz constraint. Excluding these outlying configurations, the remaining settings cluster into comparatively narrow, stable bands, indicating that the training procedure is reasonably robust across a moderate range of gradient penalty and generator learning-rate values, with $GP = 7$ and the lower learning rates (0.0002–0.001) yielding the best-behaved trajectories observed in this study. Taken together with the entangling-gate result, this suggests that the choice of entangling gate exerts a larger effect on final sample fidelity than moderate variation in either of these two optimization hyperparameters.

Conclusion

The results presented above indicate that the choice of entangling gate within the parameterized quantum circuit has a more pronounced effect on the fidelity of the generated distribution than moderate variation in either the gradient-penalty coefficient or the generator learning rate. The CNOT-based ansatz consistently achieved a lower Wasserstein distance than its CZ-based counterpart, and the hyperparameter sweeps suggest that training is comparatively robust once the gradient-penalty coefficient exceeds a threshold of approximately 9 and the generator learning rate remains below approximately 0.005; more aggressive settings in either direction produced trajectories consistent with an underconstrained or unstable critic rather than genuine improvements in sample quality.

These conclusions should nonetheless be read within the constraints under which this project was conducted. The full study was carried out over a single month, which limited the scope of both the architectural exploration and the evaluation methodology. In particular, model quality was assessed almost exclusively through the Wasserstein distance estimated from the critic, and complementary metrics such as the Maximum Mean Discrepancy (MMD) or the Fréchet Inception Distance (FID) were not systematically explored across the gate, gradient-penalty, and learning-rate sweeps. Because the Wasserstein estimate is only a faithful measure of distributional discrepancy when the critic satisfies its Lipschitz constraint, its absolute value can be misleading precisely in the unstable regimes, such as a low gradient-penalty coefficient or a high generator learning rate, that this study identifies as problematic. A distribution-level metric such as MMD, or a perceptual metric such as FID computed in a learned feature space, would provide an independent check on whether the lower Wasserstein distance observed for CNOT and for the more conservative hyperparameter settings corresponds to a genuine improvement in the visual quality of the generated digits, rather than an artifact of critic behavior.

Similarly, the generator architecture used throughout this work was restricted to a purely linear composition of parameterized quantum circuits, with no non-linear activation function inserted between the patch subgenerators or at the output stage. This design choice was largely a consequence of the limited project timeline rather than a deliberate architectural decision, and it constrains the expressive power of the generator: without an explicit source of non-linearity, the model's capacity to approximate complex, multi-modal image distributions may be fundamentally limited regardless of the entangling gate or optimization hyperparameters chosen. Introducing a non-linear activation, whether through a classical non-linear layer between quantum patches or through non-linear encoding strategies within the PQC itself, represents a natural and, in principle, impactful extension of this work.

Future work should therefore prioritize, in order of expected impact: (i) the introduction of a non-linear component into the generator architecture, (ii) a broader evaluation protocol

incorporating MMD and FID alongside the Wasserstein distance to disambiguate genuine quality improvements from critic instability artifacts, and (iii) a joint sweep over entangling gate, gradient-penalty coefficient, and learning rate, rather than the univariate sweeps conducted here, in order to characterize potential interaction effects between these factors within the time available for this project.

Bibliography

[1] S. L. Tsang, M. T. West, S. M. Erfani, and M. Usman, "Hybrid quantumclassical generative adversarial network for high-resolution image generation," *IEEE Transactions on Quantum Engineering*, vol. 4, p. 119, 2023.

[2] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial networks," 2014.

[3] L. Mescheder, A. Geiger, and S. Nowozin, "Which training methods for gans do actually converge?," 2018.

[4] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein gan," 2017.

[5] T. Karras, T. Aila, S. Laine, and J. Lehtinen, "Progressive growing of gans for improved quality, stability, and variation," 2018.

[6] S. Lloyd and C. Weedbrook, "Quantum generative adversarial learning," *Physical Review Letters*, vol. 121, July 2018.

[7] P.-L. Dallaire-Demers and N. Killoran, "Quantum generative adversarial networks," *Physical Review A*, vol. 98, July 2018.

[8] M. Benedetti, E. Lloyd, S. Sack, and M. Fiorentini, "Parameterized quantum circuits as machine learning models," *Quantum Science and Technology*, vol. 4, p. 043001, Nov. 2019.

[9] J. Jäger, F. J. Kiwit, and C. A. Riofrío, "Scaling quantum machine learning without tricks: High-resolution and diverse image generation," 2026.

[10] H.-L. Huang, Y. Du, M. Gong, Y. Zhao, Y. Wu, C. Wang, S. Li, F. Liang, J. Lin, Y. Xu, R. Yang, T. Liu, M.-H. Hsieh, H. Deng, H. Rong, C.-Z. Peng, C.-Y. Lu, Y.-A. Chen, D. Tao, X. Zhu, and J.-W. Pan, "Experimental quantum generative adversarial networks for image generation," *Physical Review Applied*, vol. 16, Aug. 2021.

[11] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," 2022.

[12] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep unsupervised learning using nonequilibrium thermodynamics," 2015.

[13] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, "Improved training of wasserstein gans," 2017.

[14] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. J. Coles, "Variational quantum algorithms," *Nature Reviews Physics*, vol. 3, p. 625644, Aug. 2021.

[15] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, "Barren plateaus in quantum neural network training landscapes," *Nature Communications*, vol. 9, Nov. 2018.

[16] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, S. Ahmed, V. Ajith, M. S. Alam, G. Alonso-Linaje, B. AkashNarayanan, A. Asadi, J. M. Arrazola, U. Azad, S. Banning, C. Blank, T. R. Bromley, B. A. Cordier, J. Ceroni, A. Delgado, O. D. Matteo, A. Dusko, T. Garg, D. Guala, A. Hayes, R. Hill, A. Ijaz, T. Isacsson, D. Ittah, S. Jahangiri, P. Jain, E. Jiang, A. Khandelwal, K. Kottmann, R. A. Lang, C. Lee, T. Loke, A. Lowe, K. McKiernan, J. J. Meyer, J. A. Montañez-Barrera, R. Moyard, Z. Niu, L. J. O'Riordan, S. Oud, A. Panigrahi, C.-Y. Park, D. Polatajko, N. Quesada, C. Roberts, N. Sá, I. Schoch, B. Shi, S. Shu, S. Sim, A. Singh, I. Strandberg, J. Soni, A. Száva, S. Thabet, R. A. Vargas-Hernández, T. Vincent, N. Vitucci, M. Weber, D. Wierichs, R. Wiersema, M. Willmann, V. Wong, S. Zhang, and N. Killoran, "Pennylane: Automatic differentiation of hybrid quantum-classical computations," 2022.

[17] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," 2019.

Appendix

1 Details on generator hyperparameters effects.

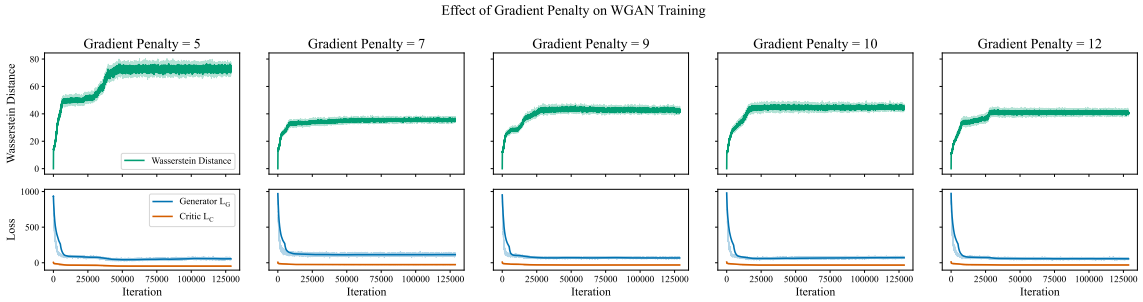


Fig. A.1: Gradient penalty effect on the training process.

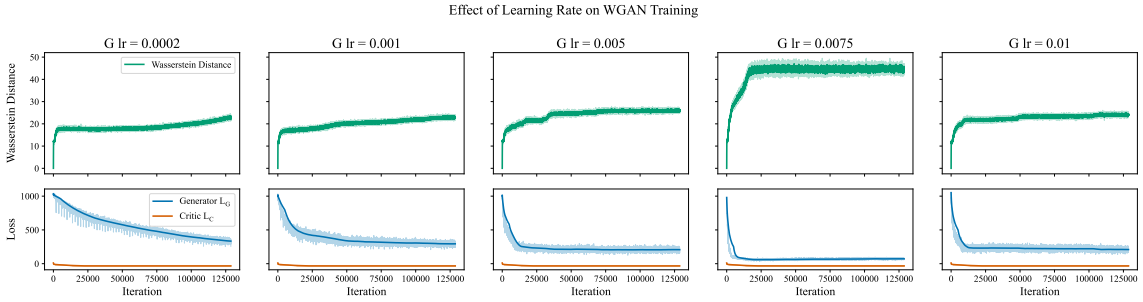


Fig. A.2: Learning rate of the generator during training.

2 Differences from the different ansatz

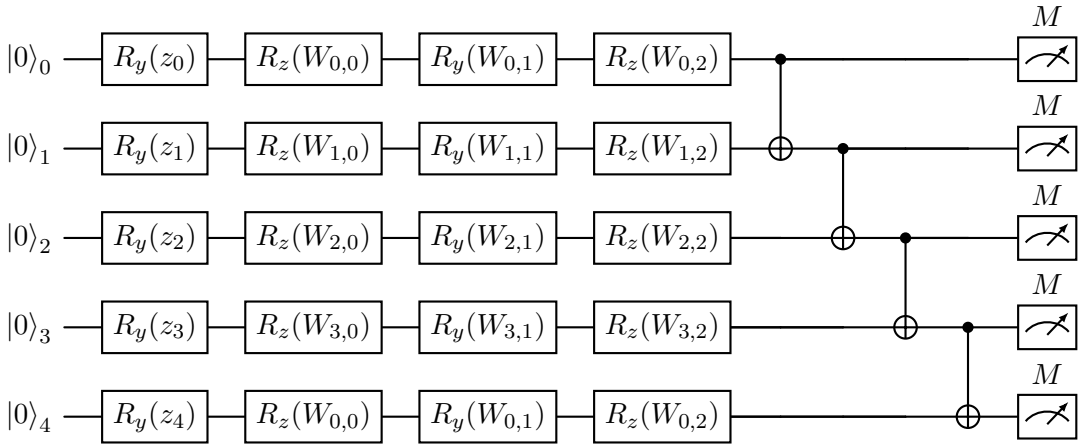


Fig. A.3: Quantum Generator circuit for 5 qubits showing the latent space mapping and a single variational layer (l). Here, z_i represents the noise input, and $W_{i,k}$ corresponds to the trainable parameters. Note that the CNOT gates are applied sequentially on neighboring qubits. Finally, the measurements yield the probability distribution.

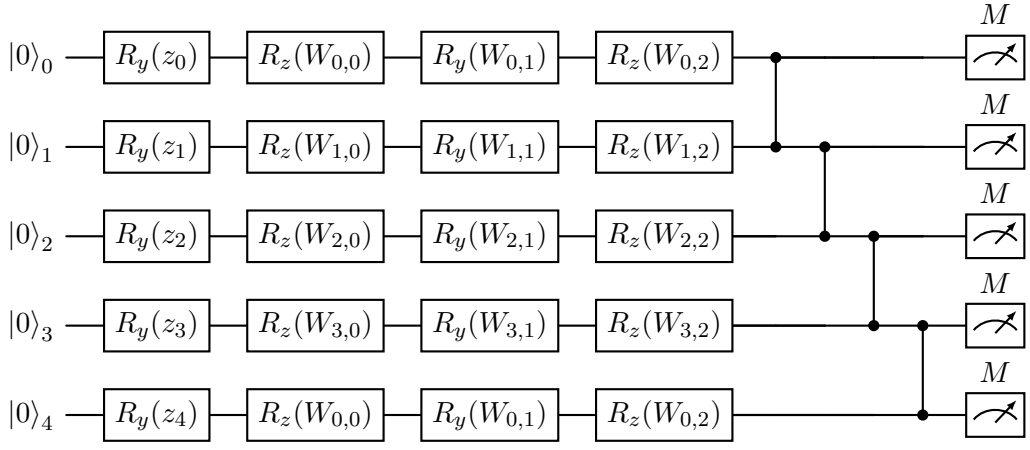


Fig. A.4: Quantum Generator circuit for 5 qubits showing the latent space mapping and a single variational layer (l). Here, z_i represents the noise input, and $W_{i,k}$ corresponds to the trainable parameters. Note that the CZ gates are applied sequentially on neighboring qubits. Finally, the measurements yield the probability distribution.

Annexe

1 Cost Functions

In hybrid quantum-classical generative adversarial architectures, the objective functions guide both the quantum generator $G_{\boldsymbol{\theta}}$ and the classical critic $D_{\boldsymbol{\phi}}$ to achieve convergence towards a Wasserstein-1 equilibrium[cite: 1].

1.1 Critic Loss with Gradient Penalty

The critic loss implements the Kantorovich-Rubinstein duality of the Wasserstein metric augmented with a gradient penalty to enforce the 1-Lipschitz condition:

$$\mathcal{L}_D(\boldsymbol{\phi}) = \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}}\left[D_{\boldsymbol{\phi}}(G_{\boldsymbol{\theta}}(\boldsymbol{z}))\right] - \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}}\left[D_{\boldsymbol{\phi}}(\boldsymbol{x})\right] + \lambda_{\text{gp}}\mathbb{E}_{\hat{\boldsymbol{x}}\sim p_{\hat{\boldsymbol{x}}}}\left[\left(\|\nabla_{\hat{\boldsymbol{x}}} D_{\boldsymbol{\phi}}(\hat{\boldsymbol{x}})\|_2 - 1\right)^2\right], \quad \text{(B.1)}$$

where $\hat{\boldsymbol{x}} = \epsilon\boldsymbol{x} + (1-\epsilon)G_{\boldsymbol{\theta}}(\boldsymbol{z})$ with $\epsilon \sim \mathcal{U}[0,1]$ defines the linear interpolation distribution $p_{\hat{\boldsymbol{x}}}$, and $\lambda_{\text{gp}} = 10.0$ is the regularization parameter.

1.2 Generator Loss with Total Variation Regularization

The quantum generator minimizes the critic's realness assessment while applying a Total Variation (TV) penalty to mitigate mosaic boundary artifacts across independently generated sub-patches[cite: 1]:

$$\mathcal{L}_G(\boldsymbol{\theta}) = -\mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}}\left[D_{\boldsymbol{\phi}}(G_{\boldsymbol{\theta}}(\boldsymbol{z}))\right] + \lambda_{\text{tv}}\mathcal{L}_{\text{tv}}(G_{\boldsymbol{\theta}}(\boldsymbol{z})), \quad \text{(B.2)}$$

where $\lambda_{\text{tv}} = 0.1$, and the anisotropic Total Variation regularizer for a 2D image $X \in \mathbb{R}^{H\times W}$ is given by:

$$\mathcal{L}_{\text{tv}}(X) = \frac{1}{H\times W}\sum_{i=1}^{H-1}\sum_{j=1}^{W-1}\left(|X_{i+1,j} - X_{i,j}| + |X_{i,j+1} - X_{i,j}|\right). \quad \text{(B.3)}$$

2 Measurement and Quantum Expectation

In Parameterized Quantum Circuits (PQCs), classical data outputs are extracted by measuring observables on the parameterized quantum state $|\psi(\boldsymbol{\theta}, \boldsymbol{z})\rangle$.

Given an initial state $|0\rangle^{\otimes N}$, the state vector is evolved under a parameterized unitary operator $U(\boldsymbol{\theta}, \boldsymbol{z})$:

$$|\psi(\boldsymbol{\theta}, \boldsymbol{z})\rangle = U(\boldsymbol{\theta}, \boldsymbol{z})|0\rangle^{\otimes N}. \quad \text{(B.4)}$$

For an observable $\hat{\mathcal{M}}$, the expected measurement value is calculated as:

$$\langle\hat{\mathcal{M}}\rangle_{\boldsymbol{\theta},\boldsymbol{z}} = \langle\psi(\boldsymbol{\theta},\boldsymbol{z})|\hat{\mathcal{M}}|\psi(\boldsymbol{\theta},\boldsymbol{z})\rangle = \mathrm{Tr}\left(\hat{\mathcal{M}}|\psi(\boldsymbol{\theta},\boldsymbol{z})\rangle\langle\psi(\boldsymbol{\theta},\boldsymbol{z})|\right). \tag{B.5}$$

For patch generation, computational basis projective measurements $\Pi_k = |k\rangle\langle k|$ are applied on data qubits conditioned on the ancilla qubit state $|q_a = 0\rangle$[cite: 1]:

$$p(k \,|\, q_a = 0) = \frac{\langle\psi(\boldsymbol{\theta},\boldsymbol{z})|\,(|0\rangle\langle 0|_a \otimes |k\rangle\langle k|_d)\,|\psi(\boldsymbol{\theta},\boldsymbol{z})\rangle}{\langle\psi(\boldsymbol{\theta},\boldsymbol{z})|\,(|0\rangle\langle 0|_a \otimes \mathbb{I}_d)\,|\psi(\boldsymbol{\theta},\boldsymbol{z})\rangle}, \quad k \in \{0,1,\ldots,2^D-1\}, \tag{B.6}$$

where $D = 4$ data qubits yield a 16-dimensional vector corresponding to a flattened 4×4 pixel patch[cite: 1].

3 Gradient Descent

Optimizing variational quantum circuits via classical gradient descent requires computing gradients of quantum expectation values with respect to circuit parameters $\boldsymbol{\theta}$.

3.1 Parameter-Shift Rule

Because quantum state intermediate superpositions cannot be directly accessed without state collapse, analytical gradients for gates generated by Pauli operators $G = e^{-i\frac{\theta}{2}P}$ (where $P^2 = \mathbb{I}$) are evaluated using the parameter-shift rule:

$$\frac{\partial\langle\hat{\mathcal{M}}\rangle}{\partial\theta_i} = \frac{\langle\hat{\mathcal{M}}\rangle\,(\theta_i + \frac{\pi}{2}) - \langle\hat{\mathcal{M}}\rangle\,(\theta_i - \frac{\pi}{2})}{2}. \tag{B.7}$$

This allows exact computation of the quantum generator gradients on quantum hardware or simulators without relying on finite-difference numerical approximations.

4 Derivation of the Post-Measurement Density Matrix

This appendix derives the expression for the density matrix obtained after a partial projective measurement on a subset of qubits, as used in Section **??** (Eq. (3.6)).

4.1 Projective Measurements on Composite Systems

Consider a pure quantum state $|\psi_i\rangle$ defined on a Hilbert space $\mathcal{H} = \mathcal{H}_A \otimes \mathcal{H}_D$, composed of an A-qubit ancilla register and a D-qubit data register, such that $N = A + D$. According to the postulates of quantum mechanics, a projective measurement on this system is described by a set of orthogonal projectors $\{P_m\}$ acting on $\mathcal{H}$, satisfying completeness $\sum_m P_m = \mathbb{I}$ and orthogonality $P_m P_{m'} = \delta_{mm'} P_m$.

When the measurement acts only on the ancilla subsystem $\mathcal{H}_A$, the corresponding projector on the full Hilbert space takes the form

$$P_m = \Pi_m \otimes \mathbb{I}_D, \tag{B.8}$$

where Π_m is a projector acting on $\mathcal{H}_A$ and $\mathbb{I}_D$ is the identity on the data register, left untouched by the measurement.

4.2 Post-Measurement State

The probability of obtaining outcome m is given by the Born rule,

$$p_m = \langle\psi_i| P_m |\psi_i\rangle = \langle\psi_i| (\Pi_m \otimes \mathbb{I}_D) |\psi_i\rangle , \tag{B.9}$$

and, conditioned on this outcome, the state of the composite system collapses to

$$|\psi_i^{(m)}\rangle = \frac{P_m |\psi_i\rangle}{\sqrt{p_m}} = \frac{(\Pi_m \otimes \mathbb{I}_D) |\psi_i\rangle}{\sqrt{\langle\psi_i| (\Pi_m \otimes \mathbb{I}_D) |\psi_i\rangle}}. \tag{B.10}$$

In the present case, the measurement of interest projects the ancilla qubits onto the all-zero state, i.e., $\Pi_0 = (|0\rangle\langle 0|)^{\otimes A}$, so that

$$|\psi_i^{(0)}\rangle = \frac{(|0\rangle\langle 0|)^{\otimes A} \otimes \mathbb{I}_D \ |\psi_i\rangle}{\sqrt{\langle\psi_i| (|0\rangle\langle 0|)^{\otimes A} \otimes \mathbb{I}_D \ |\psi_i\rangle}}. \tag{B.11}$$

4.3 Reduced Density Matrix of the Data Register

Since the goal is to describe the resulting state of the data qubits independently of the (now discarded) ancilla register, it is necessary to trace out the ancilla degrees of freedom. The density matrix of the full post-measurement state is

$$\rho^{(0)} = |\psi_i^{(0)}\rangle\langle\psi_i^{(0)}| = \frac{(|0\rangle\langle 0|)^{\otimes A} \otimes \mathbb{I}_D \ |\psi_i\rangle\langle\psi_i| \ (|0\rangle\langle 0|)^{\otimes A} \otimes \mathbb{I}_D}{\langle\psi_i| (|0\rangle\langle 0|)^{\otimes A} \otimes \mathbb{I}_D \ |\psi_i\rangle}. \tag{B.12}$$

Because $(|0\rangle\langle 0|)^{\otimes A}$ is idempotent (i.e., $\Pi_0^2 = \Pi_0$), the numerator simplifies, giving

$$\rho^{(0)} = \frac{(|0\rangle\langle 0|)^{\otimes A} \otimes \mathbb{I}_D \ |\psi_i\rangle\langle\psi_i|}{\langle\psi_i| (|0\rangle\langle 0|)^{\otimes A} \otimes \mathbb{I}_D \ |\psi_i\rangle}. \tag{B.13}$$

The reduced state of the data register is finally obtained by applying the partial trace operator Tr_A over the ancilla subsystem,

$$\rho_D = \mathrm{Tr}_A\left(\rho^{(0)}\right) = \mathrm{Tr}_A\left(\frac{(|0\rangle\langle 0|)^{\otimes A} \otimes \mathbb{I}_D \ |\psi_i\rangle\langle\psi_i|}{\langle\psi_i| (|0\rangle\langle 0|)^{\otimes A} \otimes \mathbb{I}_D \ |\psi_i\rangle}\right), \tag{B.14}$$

which recovers Eq. (3.6). The operator $\mathrm{Tr}_A(\cdot)$ is defined, for an orthonormal basis $\{|a_j\rangle\}_{j=1}^{2^A}$ of $\mathcal{H}_A$, as

$$\mathrm{Tr}_A(X) = \sum_{j=1}^{2^A} \langle a_j| X |a_j\rangle , \tag{B.15}$$

where X is any operator acting on $\mathcal{H}_A \otimes \mathcal{H}_D$. The resulting ρ_D is a valid density matrix on $\mathcal{H}_D$: it is Hermitian, positive semi-definite, and satisfies $\mathrm{Tr}(\rho_D) = 1$, and in general describes a mixed state, since the partial measurement introduces classical uncertainty over the data qubits even when the original state $|\psi_i\rangle$ was pure.